Classical NLP

Yurii Paniv

Course Instructors



Yurii Paniv
PhD Student @ UCU
Data Scientist @ Nortal

https://t.me/pehade_blog

Topics of the lecture

- Statistical and Rule-Based Methods
- Pre-deep learning era techniques (before 2013)
- Foundation for understanding modern NLP
- Practical methods still used in production systems

What is Classical NLP?

- Rule-based systems: Hand-crafted linguistic rules and patterns
- Statistical methods: Probabilistic models trained on annotated data
- Feature engineering: Manual design of text representations
- Interpretable models: Can understand why decisions are made

To summarize, find the way to hard-code rules for language.

Why Classical NLP?

- Fast, accurate enough
- Don't need millions of examples
- Interpretability: Required in legal, medical, financial domains
- Run on CPUs, edge devices
- Understand evolution of NLP methods

Classical NLP Timeline

- 1950s-1960s: Rule-based systems, regular expressions
- 1970s-1980s: Expert systems, knowledge representation
- 1990s: Statistical revolution, Hidden Markov Models
- 2000s: Maximum entropy, CRF, SVM for text

Rule-Based vs Statistical

- IF-THEN rules, regular expressions, grammars
- Learn patterns from data, probability distributions
- Combine rules with statistics for best results

Newspaper

Newspaper3k: Article scraping & curation

pypi package 0.2.8 | build unknown | coverage unknown

Inspired by requests for its simplicity and powered by lxml for its speed:

> "Newspaper is an amazing python library for extracting & curating articles." -- tweeted by Kenneth Reitz, Author of requests
>
> "Newspaper delivers Instapaper style article extraction." -- The Changelog

https://github.com/codelucas/newspaper/

Newspaper summarizer

```
def summarize(url='', title='', text='', max_sents=5):
    if not text or not title or max_sents <= 0:
        return []

    summaries = []
    sentences = split_sentences(text)
    keys = keywords(text)
    titleWords = split_words(title)

    # Score sentences, and use the top 5 or max_sents sentences
    ranks = score(sentences, titleWords, keys).most_common(max_sents)
    for rank in ranks:
        summaries.append(rank[0])
    summaries.sort(key=lambda summary: summary[0])
    return [summary[1] for summary in summaries]
```

Newspaper summarizer

```
def score(sentences, titleWords, keywords):
    """Score sentences based on different features
    """
    senSize = len(sentences)
    ranks = Counter()
    for i, s in enumerate(sentences):
        sentence = split_words(s)
        titleFeature = title_score(titleWords, sentence)
        sentenceLength = length_score(len(sentence))
        sentencePosition = sentence_position(i + 1, senSize)
        sbsFeature = sbs(sentence, keywords)
        dbsFeature = dbs(sentence, keywords)
        frequency = (sbsFeature + dbsFeature) / 2.0 * 10.0
        # Weighted average of scores from four categories
        totalScore = (titleFeature*1.5 + frequency*2.0 +
                      sentenceLength*1.0 + sentencePosition*1.0)/4.0
        ranks[(i, s)] = totalScore
    return ranks
```

Переозброєння разом з Європою: як Україна залучає західні інвестиції в "оборонку"

Claude 4 Sonnet

Україна перейшла від майже повної залежності від західної військової допомоги у 2022 році до виробництва близько 40% своїх оборонних потреб у 2025 році завдяки співпраці із західними партнерами. Західні компанії (Rheinmetall, KNDS, Northrop Grumman та інші) створюють спільні підприємства в Україні, оскільки вона стала "країною можливостей" для випробування оборонної продукції в реальних бойових умовах. ЄС запустив програму ReArm Europe з інвестиціями 800 млрд євро до 2030 року та механізм SAFE обсягом 150 млрд євро, який охоплює і Україну для спільних проєктів з країнами ЄС. "Данська модель" з інвестиціями понад $1 млрд у 2025 році та "нідерландська модель" прямого фінансування показали ефективність у підтримці українського оборонного виробництва. Попри прогрес, українська оборонна промисловість залишається недоінвестованою – з $35 млрд потенційних потужностей лише третина забезпечена контрактами, тому для повноцінного розвитку галузі потрібне масштабне фінансування та подолання бюрократичних перешкод.

Newspaper

Лише кільком із них вдалося започаткувати спільне виробництво в Україні.

Зокрема, турецька Baykar першою оголосила про будівництво заводу з виробництва БПЛА, однак проєкт досі не реалізовано.

Залученню інвесторів сприяє можливість випробовувати оборонну продукцію в реальних умовах і в реальному часі.

Програма дозволяє спрямовувати державне фінансування в оборонний сектор, здійснювати термінові й масштабні інвестиції у виробництво та мобілізувати приватний капітал.

Вона вже довела свою здатність до інновацій, виробництва та забезпечення значної частини власних оборонних потреб.

Expert systems

- IBM Watson
- Chatbots, intent classification
- Moved to GenAI-based solution in 2023

Expert systems

- Project Athena @ JPMorgan
- 35 million lines of Python code for actor-action simulation for analysis and risk management

JPMorgan already employs a lot of engineers. Across the bank, there are 40,000 engineers using the platform. Each month, they make 30,000 releases. Their productivity is up 10% on the recent past and, it's expected to increase further still: while JPMorgan invests an additional $3.3bn in technology in the CIB this year, it's also aiming for $1.5bn in efficiency gains from the technology team before 2025.

https://www.efinancialcareers.com/news/2022/05/jpmorgan-technology-jobs-and-spending

The Classical NLP Pipeline

1. Text Preprocessing: Clean and normalize input
2. Tokenization: Split into meaningful units
3. Feature Extraction: Convert text to numerical representation
4. Model Application: Classification, tagging, parsing

Languages and Classical NLP

- English: Most resources and tools available
- Morphologically rich languages: Need different approaches
- Multilingual challenges: Different scripts, grammars

Text Normalization

- Case folding: "Apple" → "apple" (but lose proper noun info)
- Number normalization: "123" → "NUM" or expand to words
- Date normalization: Various formats → standard format
- Punctuation: Keep, remove, or normalize based on task

Tokenization Challenges

- Word boundaries: "don't" → ["do", "n't"] or ["don't"]?
- Multiword expressions: "New York" as one or two tokens?
- URLs, emails, hashtags: Need special rules
- Non-space languages: Chinese, Japanese require different methods

Regular Expressions for NLP

- Pattern matching: Phone numbers, emails, URLs
- Text extraction: Pull structured data from unstructured text
- Tokenization rules: Define what counts as a token
- Limitations: Can't handle recursive structures, context

Practical Regex Patterns

```
# Email pattern
email = r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b'
# URL pattern
url = r'http[s]?://(?:[a-zA-Z]|[0-9]|[$-_@.&+]|[!*\\(\\),]|(?:%[0-9a-fA-F][0-9a-fA-F]))+'
# Phone number
phone = r'(\+\d{1,3}[-.\s]?)?\(?\d{3}\)?[-.\s]?\d{3}[-.\s]?\d{4}'
```

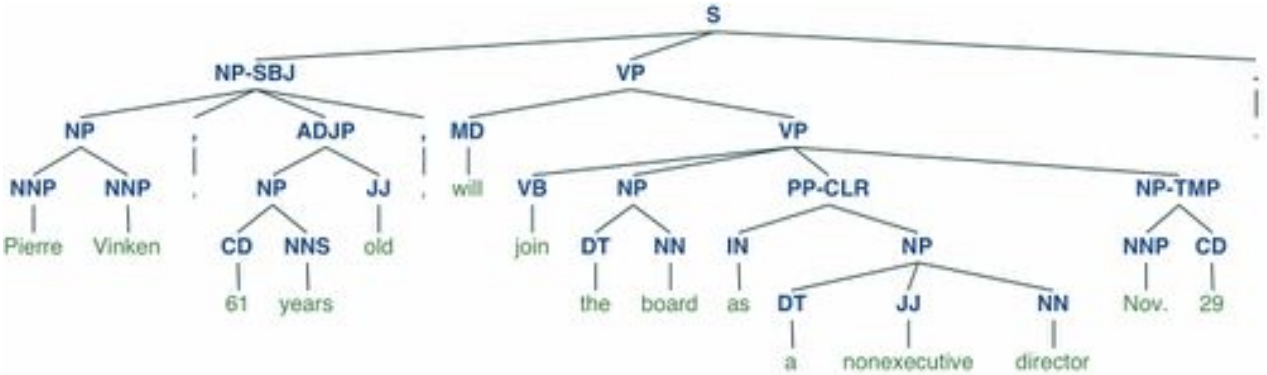
Sentence Segmentation

- Simple rule: Split on .!?
- Problems: Abbreviations (Dr.), decimals (3.14)
- Solutions: Exception lists, machine learning approaches
- Tools: NLTK punkt tokenizer, spaCy

NLTK

Natural Language Toolkit

NLTK is a leading platform for building Python programs to work with human language data. It provides easy-to-use interfaces to **over 50 corpora and lexical resources** such as WordNet, along with a suite of text processing libraries for classification, tokenization, stemming, tagging, parsing, and semantic reasoning, wrappers for industrial-strength NLP libraries, and an active **discussion forum**.



SpaCy

Edit the code & try spaCy spaCy v3.7 · Python 3 · via Binder

```
# pip install -U spacy
# python -m spacy download en_core_web_sm
import spacy

# Load English tokenizer, tagger, parser and NER
nlp = spacy.load("en_core_web_sm")

# Process whole documents
text = ("When Sebastian Thrun started working on self-driving cars at "
        "Google in 2007, few people outside of the company took him "
        "seriously. “I can tell you very senior CEOs of major American "
        "car companies would shake my hand and turn away because I wasn’t "
        "worth talking to,” said Thrun, in an interview with Recode earlier "
        "this week.")
doc = nlp(text)

# Analyze syntax
print("Noun phrases:", [chunk.text for chunk in doc.noun_chunks])
print("Verbs:", [token.lemma_ for token in doc if token.pos_ == "VERB"])

# Find named entities, phrases and concepts
for entity in doc.ents:
    print(entity.text, entity.label_)
```

RUN

Features

- Support for **75+ languages**
- **84 trained pipelines** for 25 languages
- Multi-task learning with pretrained **transformers** like BERT
- Pretrained **word vectors**
- State-of-the-art speed
- Production-ready **training system**
- Linguistically-motivated **tokenization**
- Components for **named entity** recognition, part-of-speech tagging, dependency parsing, sentence segmentation, **text classification**, lemmatization, morphological analysis, entity linking and more
- Easily extensible with **custom components** and attributes
- Support for custom models in **PyTorch**, **TensorFlow** and other frameworks
- Built in **visualizers** for syntax and NER
- Easy **model packaging**, deployment and

Stemming Algorithms

Stemming is a process of reducing words to their root form.

- Porter Stemmer (1980): Series of rewrite rules
- Snowball Stemmer: Improved Porter for multiple languages
- Lancaster Stemmer: More aggressive, sometimes over-stems
- Use cases: Information retrieval, reduce vocabulary size

```
Stemmer Comparison:
================================================
Word            Porter      Lancaster    Snowball
------------------------------------------------
fairly          fairli      fair         fair
sportingly      sportingli  sport        sport
generously      gener       gen          generous
organization    organ       org          organ
civilization    civil       civil        civil
specialization  special     spec         special
running         run         run          run
swimming        swim        swim         swim
beginning       begin       begin        begin
studies         studi       study        studi
flies           fli         fli          fli
cities          citi        city         citi
```

Source

Porter Stemmer Rules

Step 1a: Replace plurals

SSES → SS (caresses → caress)

IES → I (ponies → poni)

S → ∅ (cats → cat)

Step 2: Replace suffixes

ATIONAL → ATE (relational → relate)

IZER → IZE (digitizer → digitize)

What can go wrong here? What would happen with “rational”?

Lemmatization Process

- Morphological analysis: Understand word structure
- Dictionary lookup: Find base form
- Part-of-speech dependent: "better" (ADJ) → "good"
- Requires extensive dictionary

Text Cleaning Decisions

- Task-dependent: Sentiment analysis keeps punctuation
- Domain-specific: Medical texts need special handling
- Language-specific: German compound words, Arabic diacritics
- Evaluation: Always test impact on downstream task

Stop Word Handling

Stop words are the words in a stop list (or stoplist or negative dictionary) which are filtered out ("stopped") before or after processing of natural language data (i.e. text) because they are deemed to have little semantic value or are otherwise insignificant for the task at hand.

“The”, “and”, “in” and so on

- Standard lists: NLTK, spaCy provide for many languages
- Frequency-based: Top N most frequent words
- TF-IDF based: Words with low IDF scores
- Domain adaptation: "patient" might be stopword in medical texts

Feature Extraction Overview

- Bag of Words: Simple word counts
- TF-IDF: Weight by importance
- N-grams: Capture local context
- Manual features: Length, punctuation, capitalization

From Text to Vectors

```
# Document-term matrix example
docs = ["the cat sat", "the dog sat", "cats and dogs"]
# Vocabulary: [and, cat, cats, dog, dogs, sat, the]
# Matrix:
# [[0, 1, 0, 0, 0, 1, 1],  # "the cat sat"
#  [0, 0, 0, 1, 0, 1, 1],  # "the dog sat"
#  [1, 0, 1, 0, 1, 0, 0]]  # "cats and dogs"
```

TF-IDF

$$w_{x,y} = tf_{x,y} \times \log\left(\frac{N}{df_x}\right)$$

TF-IDF

Term x within document y

$tf_{x,y}$ = frequency of x in y

df_x = number of documents containing x

N = total number of documents

TF-IDF

For 2 documents, A and B

Word	TF		IDF	TF*IDF	
	A	B		A	B
The	1/7	1/7	log(2/2) = 0	0	0
Car	1/7	0	log(2/1) = 0.3	0.043	0
Truck	0	1/7	log(2/1) = 0.3	0	0.043
Is	1/7	1/7	log(2/2) = 0	0	0
Driven	1/7	1/7	log(2/2) = 0	0	0
On	1/7	1/7	log(2/2) = 0	0	0
The	1/7	1/7	log(2/2) = 0	0	0
Road	1/7	0	log(2/1) = 0.3	0.043	0
Highway	0	1/7	log(2/1) = 0.3	0	0.043

Source

Challenge: Ambiguity

“italian reservation in palo alto”

- **Interpretation** : *Book a table at an Italian restaurant in Palo Alto.*
  >>> `$Cuisine reservation in $Location`

“indian reservation in montana”

- **Interpretation**: *A Native American reservation located in Montana.*

“mission bike directions”

- **Interpretation**: *Directions to Mission District by bike.*
  >>> `$Location $TransportationMode directions`

“mission bicycle directions”

- **Interpretation**: *Directions to a bicycle shop called “Mission Bicycles.”*

Language Modeling Task

- Goal: Predict next word given context
- Applications: Speech recognition, machine translation, predictive typing, spell checker
- Evaluation: Perplexity (lower is better)
- Foundation for many NLP tasks

N-gram Definition

- Unigram: P(word) - individual word probabilities
- Bigram: P(word|previous) - one word context
- Trigram: P(word|prev2, prev1) - two word context
- Trade-off: More context vs data sparsity

Maximum Likelihood Estimation

- Count n-grams in training corpus
- P(w2|w1) = Count(w1, w2) / Count(w1)
- Example: P(cat|the) = Count("the cat") / Count("the")
- Problem: Zero counts for unseen n-grams

Bigram Example

Corpus: "the cat sat. the cat ate. the dog sat."

Counts:

"the cat": 2, "cat sat": 1, "cat ate": 1

"the dog": 1, "dog sat": 1

P(cat|the) = 2/3, P(dog|the) = 1/3

P(sat|cat) = 1/2, P(ate|cat) = 1/2

N-gram Implementation

```
from collections import defaultdict, Counter


class BigramModel:
    def __init__(self):
        self.bigrams = defaultdict(Counter)
        self.unigrams = Counter()

    def train(self, text):
        words = text.split()
        for i in range(len(words) - 1):
            self.bigrams[words[i]][words[i + 1]] += 1
            self.unigrams[words[i]] += 1
```

Generating Text with N-grams

```
import random


def generate(self, start_word, length=20):
    words = [start_word]
    for _ in range(length):
        current = words[-1]
        # Get probability distribution
        next_words = self.bigrams[current]
        if not next_words:
            break
        # Sample from distribution
        word = random.choices(
            list(next_words.keys()), weights=list(next_words.values())
        )[0]
        words.append(word)
    return " ".join(words)
```

Let’s train a language model! (using “The City” by Pidmohylny)

Model trains in 1 second!

Степан подався на обличчі в сліди предків. Вчора він, з гірким махорочним пилом. Все це має? Нічого, крім виступу запрошених артистів, улаштувати й гаманця, бо життя невидним, потайним, але вибирати не зрозумів, що швидко складаючи рядки, непомітно день у природу, де сунулась вперед і на нього симпатію, бо виглядала його, де саме ім’я і ретушування, яка його з босяцькою щирістю розповів — конче потрібні! Але лихо його думку, не сподівалась уже виконувати приписи закону, то вільні були жадібними очима трамвай, що сам себе до висновку, що сиділи вздовж будинків, таких крихітних і спізнав. Підупала від віку тополя чудно та наради, що

The Markov Property

- Limited history: Only recent context matters
- Independence: $P(w_n|w_1...w_{n-1}) \approx P(w_n|w_{n-k}...w_{n-1})$
- Enables tractable computation
- Limitation: Loses long-range dependencies

Backoff and Interpolation

- Backoff: Use lower-order model if higher-order unseen
- If no trigram, use bigram; if no bigram, use unigram
- Interpolation: Weighted combination of all orders
- $P(w3|w1,w2) = \lambda_3 P_3(w3|w1,w2) + \lambda_2 P_2(w3|w2) + \lambda_1 P_1(w3)$

Perplexity Calculation

- Perplexity = 2^{-avg log P(words)}
- Lower perplexity = better model
- Typical values: 50-500 for news text (LLMs can go as low as 1.5)
- Interpretation: Average branching factor

Character-level Models

- Model character sequences instead of words
- Advantages: Smaller vocabulary, handles OOV words
- Applications: Text generation, spelling correction
- Trade-off: Longer sequences, harder to learn

Class-based N-grams

- Group words into classes: Monday/Tuesday → WEEKDAY
- Reduces sparsity: P(WEEKDAY|on) instead of P(Monday|on)
- Two-stage model: P(class|history) × P(word|class)
- Examples: Numbers, dates, named entities

N-gram Applications

- Spelling Correction: Find most likely word given context
- Speech Recognition: Language model × acoustic model
- Machine Translation: Target language fluency
- Text Prediction: Smartphone keyboards

N-gram Limitations

- Fixed context window: Can't model long dependencies
- Data sparsity: Most n-grams never seen
- Storage: N-grams grow exponentially with n
- No generalization: "red car" doesn't help with "blue car"

KenLM

Fast language modelling

Used as recently as in Nemotron-CC paper for state-of-the-art filtering with perplexity

Used in Speech Recognition models for fixing mistakes

https://github.com/kpu/kenlm?tab=readme-ov-file

Evaluating Language Models

- Held-out perplexity: Test on unseen data
- Application metrics: WER for speech, BLEU for MT
- Human evaluation: Fluency, coherence

Dictionary-Based Approaches

- Lookup methods: Match text against predefined lists
- Applications: Named entity recognition, sentiment analysis
- Advantages: High precision, interpretable
- Limitations: Low recall, maintenance overhead

Building Gazetteers

- Sources: Wikipedia, government databases, domain resources
- Organization: Hierarchical (city → state → country)
- Maintenance: Regular updates, version control
- Ambiguity: "Apple" (company) vs "apple" (fruit)
- Hard to scale

Gazetteer Approach

```python
class Gazetteer:
    def __init__(self):
        self.entities = {
            "PERSON": set(["John Smith", "Mary Johnson"]),
            "LOCATION": set(["New York", "London"]),
            "ORGANIZATION": set(["Apple Inc", "Google"]),
        }

    def tag(self, text):
        for entity_type, entity_set in self.entities.items():
            for entity in entity_set:
                if entity in text:
                    yield (entity, entity_type)
```

Pattern-Based Extraction

- Regular expressions: Phone numbers, dates, emails
- Template patterns: "X was born in Y" → (X, birthplace, Y)
- Syntactic patterns: NP "such as" NP_list
- Cascaded patterns: Apply rules in sequence

Text: "European countries such as France, Germany, and Spain have strong economies."

Pattern matching:

- NP_1 = "European countries"

- Signal = "such as"

- NP_list = "France", "Germany", "and Spain"

Extracted relationships:

- France is-a European country

- Germany is-a European country

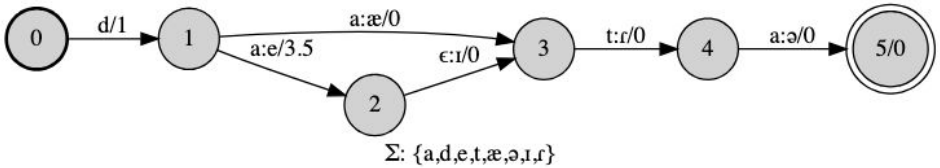
- Spain is-a European country

Finite State Transducers

- Model regular languages: Dates, numbers, addresses
- Deterministic (DFA) vs Non-deterministic (NFA)
- Efficient recognition: O(n) time complexity
- Limitations: Can't handle nested structures

https://github.com/mhulden/pyfoma

```
from pyfoma import FST
fst = FST.re("d a:(æ<1>|(eɪ)<4.5>) (ta):(ɾə)")
fst.view()
```



Context-Free Grammars

- Model hierarchical structure: S → NP VP
- Parse trees: Show syntactic structure
- Chomsky Normal Form: Binary rules only
- Applications: Syntactic parsing, information extraction

CFG Example

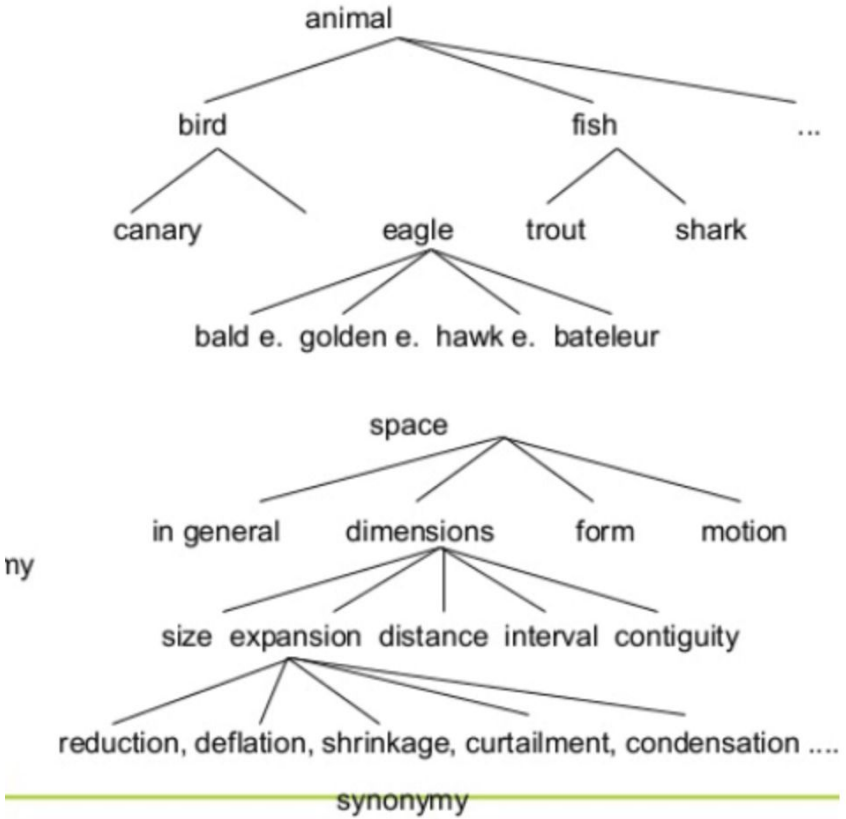
```
Grammar:
  S → NP VP
  NP → Det N | NP PP
  VP → V NP | VP PP
  PP → P NP
  Det → the | a
  N → cat | dog | mat
  V → sat | saw
  P → on | in

Parse: "the cat sat on the mat"
  S
  ├── NP (the cat)
  └── VP
      ├── V (sat)
      └── PP
          ├── P (on)
          └── NP (the mat)
```

Lexical Resources

- WordNet: Semantic network of word senses
- FrameNet: Semantic frames and roles
- VerbNet: Verb classes and alternations
- PropBank: Predicate-argument structures

WordNet Structure

- Synsets: Sets of synonymous words
- Relations: Hypernymy, meronymy, antonymy
- Word senses: "bank" has financial and river senses
- Applications: WSD, similarity, query expansion



Source: https://phdtopic.com/wordnet-projects/

Rule-Based Systems

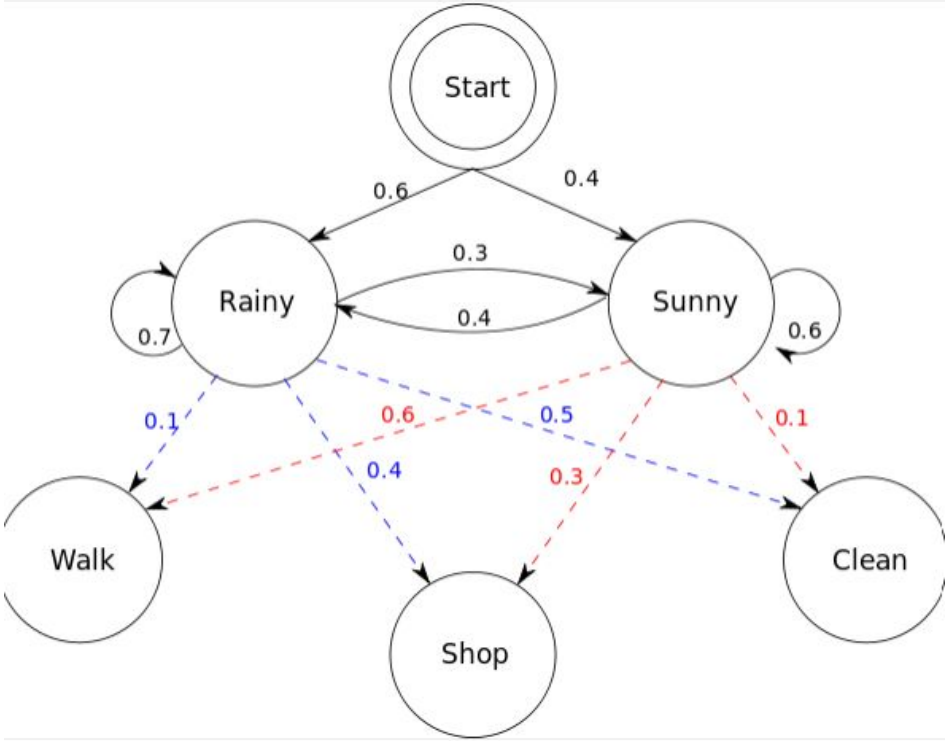
- Expert systems: Encode linguistic knowledge
- Transformation rules: Error correction, normalization
- Pros: Precise control, no training data needed
- Cons: Hard to maintain

Combining Rules and Statistics

- Rules for high-precision extraction
- Statistics for coverage and ranking
- Example: Gazetteer + context classifier
- Best of both worlds approach

Hidden Markov Models

- States: Hidden variables (e.g., POS tags)
- Observations: Visible variables (e.g., words)
- Transitions: P(state_i | state_{i-1})
- Emissions: P(observation | state)



HMM for POS Tagging

States: {NN, VB, DT, ...}

Observations: {cat, sat, the, ...}

Transition: P(NN|DT) = 0.6 # determiner → noun

Emission: P("cat"|NN) = 0.002 # noun → "cat"

Task: Find most likely tag sequence given words

Viterbi Algorithm

- Dynamic programming for HMM decoding
- Find most likely state sequence
- Time complexity: $O(n \times |states|^2)$
- Beam search: Approximate for efficiency

Forward-Backward Algorithm

- Calculate marginal probabilities
- Forward: P(observation_1...i, state_i)
- Backward: P(observation_{i+1}...n | state_i)
- Used for training with EM

Naive Bayes Classification

- Assumption: Features independent given class
- Simple but effective for text classification
- Multinomial: Word counts
- Bernoulli: Word presence/absence

We’ll be working on this in detail on practice session.

Decision Trees for Text

- Binary splits on word presence/absence
- Can capture non-linear relationships
- Feature selection built-in
- Interpretable but often overfit

Random Forests

- Ensemble of decision trees
- Bootstrap samples + random features
- Reduces overfitting
- Good for mixed feature types

Support Vector Machines

- Linear SVMs work well for text
- High-dimensional space (thousands of features)
- Maximum margin principle
- Kernel trick rarely helps for text

Clustering Algorithms

- K-means: Fast, spherical clusters
- Hierarchical: Dendrogram structure
- DBSCAN: Arbitrary shapes, outliers
- Applications: Document organization, topic discovery

Topic Modeling Overview

- Discover and extract hidden topics in documents
- Unsupervised learning
- Applications: Document browsing, summarization
- Methods: LSA, pLSA, LDA
- Then you can compare topics between two documents and calculate their coreference

More to dive:
https://www.geeksforgeeks.org/nlp/topic-modeling-using-latent-dirichlet-allocation-lda/

Evaluation Metrics

- Classification: Accuracy, precision, recall, F1
- Sequence labeling: Token-level vs entity-level
- Language models: Perplexity, cross-entropy
- Clustering: Silhouette, purity, NMI

Cross-Validation

- K-fold: Split data into k parts
- Stratified: Maintain class distribution
- Leave-one-out: For small datasets
- Time-based: For temporal data

Ensemble Methods

- Voting: Combine predictions
- Stacking: Meta-classifier on base models
- Boosting: Sequential focus on errors
- When to use: Improve robustness

Use cases for Classical NLP

- Spam filters: 99%+ accuracy with simple features
- Search engines: TF-IDF still core ranking signal
- Machine translation: Phrase-based SMT dominated until 2016
- Information extraction: Rule-based systems in production

When Classical Methods Excel

- Limited training data: <10K examples
- Interpretability required: Legal, medical domains
- Real-time constraints: Embedded devices
- Domain-specific: Specialized vocabularies

Classical + Modern Hybrid

- Feature engineering + neural models
- Rules for high precision + ML for recall
- Classical preprocessing + deep learning
- Interpretable features for model debugging

Q&A